

Makerspace Database Design – Detailed Documentation

This document provides a clear, methodical explanation of the Makerspace Management System database structure.

It explains each entity, why it exists, how it relates to others, and which rules or mechanisms enforce data consistency.

1. Overview

The system is designed around PostgreSQL with the following core entities:

users, tools, projects, bookings, inventory_items, inventory_transactions, trainings, attachments, and audit_logs.

Each entity has a UUID as its primary key and connects to others to reflect real makerspace workflows.

2. Entities and Relationships

users

- **Primary Key:** `user_id` (UUID)
 - **Fields:** `full_name`, `email` (unique), `role` (ENUM: *admin, staff, member, visitor*), `staff_type` (*full_time | intern*), `password_hash`, `phone_number`, `created_by`, `created_at`, `updated_at`.
 - **Relations:**
 - `created_by` → `users` (`user_id`) (who created the account)
 - One user → many **projects, bookings, maintenance_logs, inventory_transactions, trainings, and audit_logs.**
 - **Notes:**
 - `staff_type` is only valid when `role = 'staff'`.
 - Admins are the only ones allowed to register staff accounts.
-

tools

- **Primary Key:** `tool_id` (UUID)
- **Fields:** `name`, `serial_number`, `category`, `status`, `location`, `last_maintenance`, `maintenance_interval_days`, `is_active`.

- **Relations:**
 - One tool → many **tool_maintenance_logs**, **bookings**, **attachments**.
-

tool_maintenance_logs

- **Primary Key:** `log_id` (UUID)
 - **Fields:** `tool_id` (FK), `performed_by` (FK → **users**), `performed_at`, `notes`, `next_due`.
 - **Relations:**
 - Many logs → one tool.
 - Many logs → one user (staff who performed maintenance).
-

projects

- **Primary Key:** `project_id` (UUID)
 - **Fields:** `title`, `description`, `owner_id` (FK → **users**), `status`, `start_date`, `end_date`.
 - **Relations:**
 - One project → many **project_participants**, **bookings**, **attachments**.
-

project_participants (join table)

- **Primary Key:** (`project_id`, `user_id`)
 - **Fields:** `role_in_project`, `joined_at`.
 - **Relations:**
 - Many-to-many between **users** and **projects**.
-

bookings

- **Primary Key:** `booking_id` (UUID)
- **Fields:** `tool_id` (FK), `user_id` (FK), `project_id` (FK), `start_ts`, `end_ts`, `status`, `purpose`, `created_at`, `updated_at`.
- **Relations:**
 - Many bookings → one **tool**, one **user**, optional **project**.
- **Rules:**
 - `end_ts > start_ts`.
 - Prevent overlapping bookings for the same tool (checked in app logic or DB exclusion constraint).

inventory_items

- **Primary Key:** `item_id` (UUID)
 - **Fields:** `name`, `sku`, `unit`, `quantity`, `threshold`, `location`, `supplier`, `is_active`.
 - **Relations:**
 - One `inventory_item` → many **inventory_transactions**.
 - **Explanation:**
 - Tracks all tools and consumables.
 - Includes `quantity` and `threshold` to help flag low-stock items.
-

inventory_transactions

- **Primary Key:** `txn_id` (UUID)
 - **Fields:** `item_id` (FK), `performed_by` (FK → `users`), `txn_type` (ENUM: *addition*, *consumption*, *adjustment*, *transfer*), `quantity`, `notes`, `occurred_at`.
 - **Use:**
 - Keeps an audit of every stock change.
 - A database function ensures `quantity` never becomes negative.
-

trainings

- **Primary Key:** `training_id` (UUID)
 - **Fields:** `title`, `description`, `instructor_id` (FK → `users`), `scheduled_at`, `duration_minutes`, `capacity`.
 - **Relations:**
 - One `training` → many **training_participants**.
-

training_participants (join table)

- **Primary Key:** (`training_id`, `user_id`)
 - **Fields:** `registered_at`, `attended` (boolean).
 - **Use:** manage enrollments and attendance tracking.
-

attachments

- **Primary Key:** `attachment_id` (UUID)
 - **Fields:** `owner_table`, `owner_id`, `filename`, `file_url`, `uploaded_by`, `uploaded_at`.
 - **Explanation:**
 - A polymorphic attachment system for flexibility (links to **projects**, **tools**, or **users**).
 - Simplifies file management and avoids redundant uploads.
 - Application logic enforces correct linking.
-

audit_logs

- **Primary Key:** `audit_id` (UUID)
 - **Fields:** `user_id` (FK), `event_type`, `event_description`, `occurred_at`.
 - **Use:** records important system or user events for traceability.
-

3. Key Database Mechanisms

- **UUID primary keys:** secure, unique across distributed systems.
 - **ENUM types:** enforce valid roles and statuses.
 - **Timestamp triggers:** auto-maintain `created_at` and `updated_at`.
 - **CHECK constraints:** enforce data rules (e.g., valid booking times).
 - **Indexes:** optimize lookups on `email`, `role`, and `status`.
-

4. Enforcement Rules

Database-level (strong invariants):

- Non-null foreign keys.
- `CHECK → end_ts > start_ts`.
- `CHECK → staff_type valid only when role='staff'`.
- `UNIQUE(email)` constraint.
- Quantity cannot go negative (via transaction function).

Application-level (business logic):

- Admin-only staff registration (checked in Spring Boot).
 - Prevent booking overlaps via query or DB exclusion constraint.
 - Handle approval flows and status transitions in code.
-

5. Additional Clarifications

inventory_items Field Explanation

Manages all tools, equipment, and consumables with fields for item name, category, quantity, supplier, and storage location.

Ensures every asset is trackable, up-to-date, and usable.

attachments Table Purpose

Centralized table for managing file uploads (images, project files, user documents).

Prevents redundancy and simplifies access control.

phone_number Field in Users

Added for better communication and verification.

Required for all user registrations and supports future two-factor authentication.

6. Future Enhancements

- **Docker + Jenkins integration** for full CI/CD automation.
- **Advanced analytics dashboards** for resource usage and project statistics.
- **Automated maintenance scheduling** and tool availability predictions.